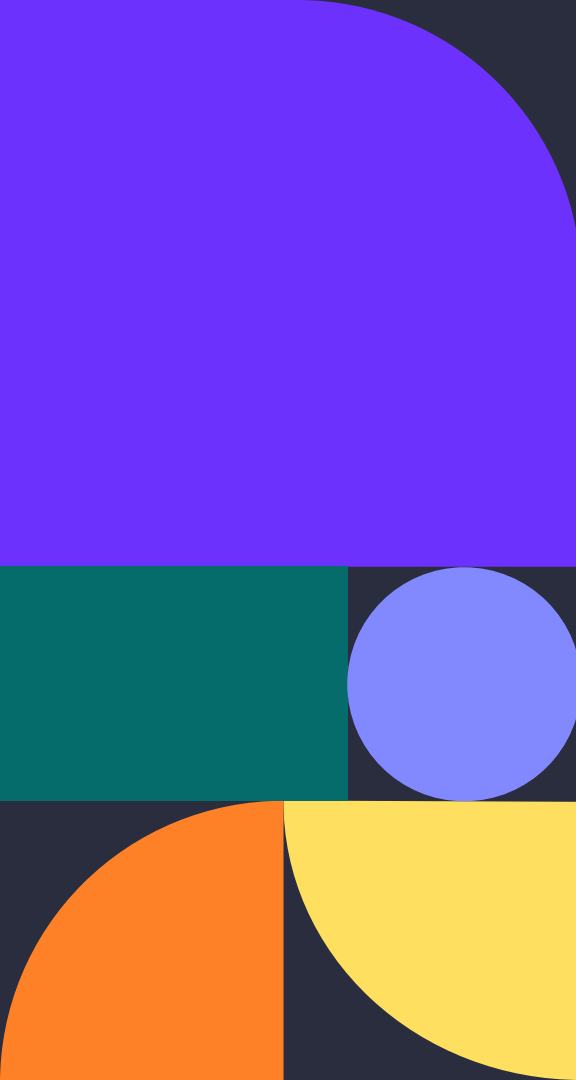




Translating Natural Language to Linear Temporal Language using LLMs

Atul Bhat

What is LTL?

LTL is a branch of the broader logic tree, which is distinct from the others in how it views time. Time is viewed as a sequence of states, in which each state has facts that are true or false, allowing rules to be upheld not just instantaneously, but over time.

Let's take an analogy of an autonomous vehicle.

There are 4 key operators that differentiate LTL from other logics:

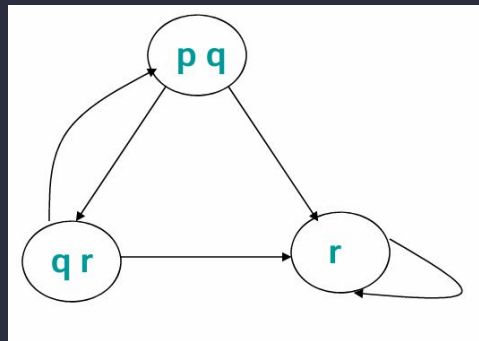
X (Next): If an alarm is triggered, the system shuts down in the **next** second

F (Finally): If I press the power button, the computer will **eventually** boot up.

G (Globally): The robot must **always** avoid hitting pedestrians.

U (Until): Keep the indicator light blinking **until** the turn is completed.

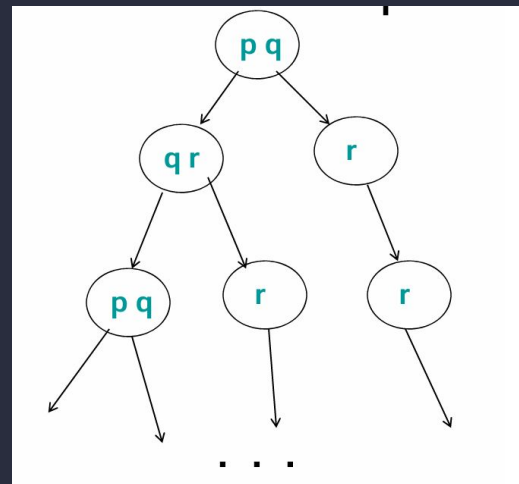
Safety vs. Liveness Properties



Safety Properties

Defining a boundary that the system is *never* allowed to cross

Finite-length error trace: To catch a bug here, you can prove it instantly with a short video clip (e.g. If a self-driving car hits a trash can; you don't need to watch the car drive for the rest of the tape to prove it failed. You just hit pause at the exact second of the crash)



Liveness Properties

Ensure the system actually makes progress and doesn't just stagnantly sit there

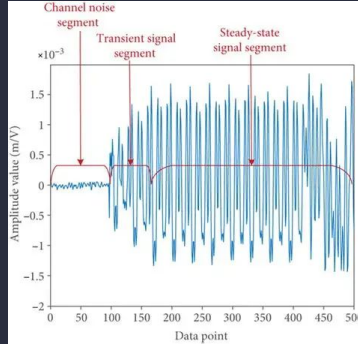
Infinite-length error trace: To catch a bug here, if you send a text message and it hasn't arrived after 5 seconds, has the system broken the rule? 1 day? 1 year? The only way you can determine it is if you look at the timeline infinitely.

Real Life Examples



Processor Coordination

Mandates that "no more than one processor has a cache line in write mode" to prevent data from being corrupted by simultaneous updates.

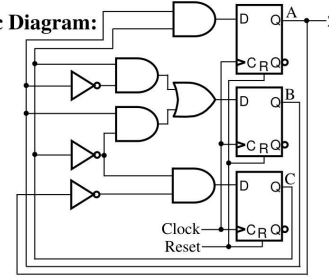


Signal Response

Guarantees that "the grant signal must be asserted at some time after the request," ensuring the system never permanently ignores a component.

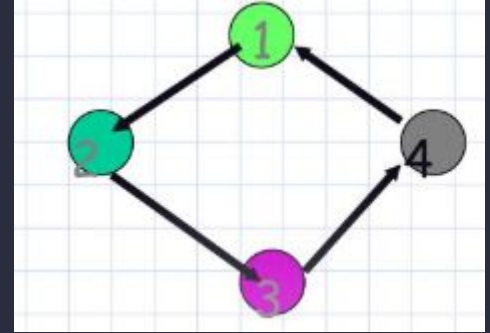
Example 2: Sequential Circuit Analysis

Logic Diagram:



Sequential Logic

Dictates that a "request stays asserted *until* the acknowledge signal is received," locking the hardware state in place until communication is formally verified.



Deadlock Prevention

Evaluates the entire state transition graph to prove the absolute "absence of deadlock," ensuring a concurrent system never enters a frozen state where no further execution paths can move forward.

Linear vs. Branching Temporal Logic

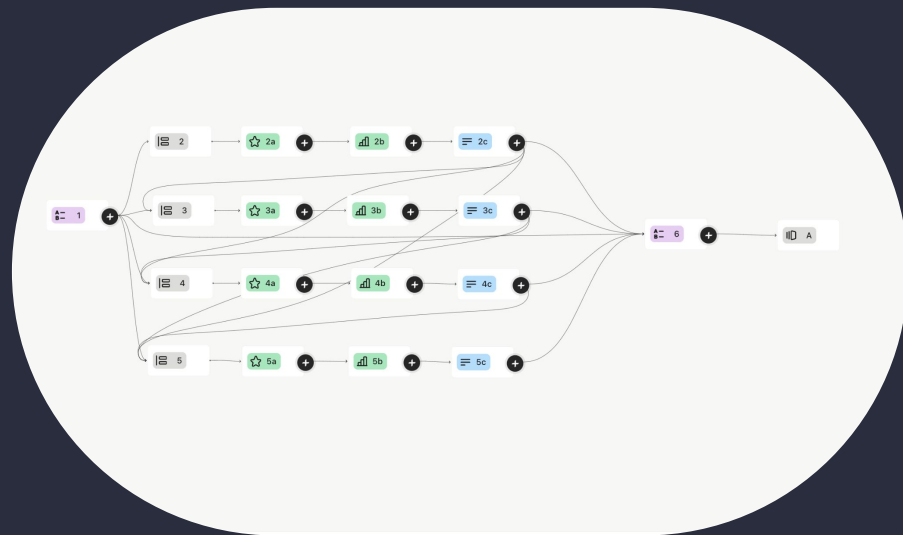
Linear Temporal Logic (LTL) looks at one single line of steps that a computer takes. It only checks what happens along that one straight timeline from start to finish.

Branching Tree Logic (CTL) looks at all the different choices a computer can make at the exact same time. Because a computer can choose different paths at any step, the choices spread out into a massive tree of paths.

Since Tree Logic deals with many choices, it also has two distinct rules:

A (All Paths): This rule means that something must happen on **every single path** that comes out of that step.

E (Exists a Path): This rule means that something only needs to happen on **at least one path** coming out of that step.





Complexity vs. Usability

LTL rules is hard work for a computer. As your rules get longer or more detailed, the math scales exponentially, meaning the system quickly needs a massive amount of computing power to check them. On the flip side, checking CTL is much easier on a machine, scaling linearly so it stays light and fast even with longer expressions.

However, when you look at the actual system under test rather than just the rule itself, both methods are on equal footing. They both scale linearly based on the overall size of the system's state graph. Even though LTL is a hogs a ton of resources for computers, engineers still heavily prefer it. It is just far more natural for humans to think about and write down, making it more popular.

Thank you